Énumérations en Swift

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'une énumération ?

Une énumération permet de représenter un ensemble limité de valeurs possibles.

Swift utilise :

```
enum
```

Exemple :

```
enum EtatConnexion {
    case connecte
    case deconnecte
    case connexionEnCours
}
```

Une variable peut alors contenir uniquement l'une de ces valeurs.

2. Utiliser une énumération

```
var etat: EtatConnexion = .deconnecte
```

Puis :

```
etat = .connecte
```

Comme Swift connaît déjà le type, on peut écrire :

```
.connecte
```

au lieu de :

```
EtatConnexion.connecte
```

3. Pourquoi utiliser enum ?

Sans énumération :

```
var etat = "connecte"
```

On pourrait accidentellement écrire :

```
etat = "conecte"
etat = "connected"
etat = "abc"
```

Avec :

```
enum EtatConnexion {
    case connecte
    case deconnecte
}
```

seules les valeurs prévues sont autorisées.

4. enum avec switch

Les énumérations fonctionnent particulièrement bien avec :

```
switch
```

Exemple :

```
enum EtatConnexion {
    case connecte
    case deconnecte
    case connexionEnCours
}

let etat = EtatConnexion.connecte

switch etat {
case .connecte:
    print("Appareil connecté")
case .deconnecte:
    print("Appareil déconnecté")
case .connexionEnCours:
    print("Connexion en cours...")
}
```

5. Avantage avec switch

Swift vérifie que tous les cas sont traités.

Si l'énumération contient :

```
case connecte
case deconnecte
case connexionEnCours
```

le switch doit normalement gérer les trois possibilités. Cela réduit les oublis dans le code.

6. Exemple — État d'un téléchargement

```
enum EtatTelechargement {
    case attente
    case enCours
    case termine
    case erreur
}
```

Utilisation :

```
var etat = EtatTelechargement.attente
```

Puis :

```
etat = .enCours
```

7. Plusieurs case sur une même ligne

On peut écrire :

```
enum TypeNotification {
    case message
    case batterie
    case systeme
}
```

ou :

```
enum TypeNotification {
    case message, batterie, systeme
}
```

Les deux syntaxes sont valides.

8. Valeurs brutes — rawValue

Une énumération peut associer une valeur simple à chaque cas.
Exemple :

```
enum Priorite: Int {
    case faible = 1
    case normale = 2
    case haute = 3
}
```

Utilisation :

```
let priorite = Priorite.haute
print(priorite.rawValue)
```

Résultat :

```
Console
```

```
3
```

9. Exemple avec String

```swift
enum Langue: String {
    case francais = "fr"
    case anglais = "en"
}
```

Puis :

```swift
let langue = Langue.francais
print(langue.rawValue)
```

Résultat :

```
Console
fr
```

Cela peut être utile pour représenter des valeurs utilisées dans une configuration.

10. Créer un enum à partir d'un rawValue

Supposons :

```swift
enum Langue: String {
    case francais = "fr"
    case anglais = "en"
}
```

On peut écrire :

```swift
let langue = Langue(rawValue: "fr")
```

Le résultat est un optionnel :

```swift
Langue?
```

car la valeur peut être invalide.

11. Exemple — Validation d'un rawValue

```swift
if let langue = Langue(rawValue: "fr") {
    print(langue)
} else {
    print("Langue inconnue")
}
```

Une valeur comme :

```
Langue(rawValue: "xx")
```

retournera :

```
nil
```

12. Valeurs associées

Les enum Swift peuvent également transporter des informations.

Exemple :

```
enum ResultatTelechargement {
    case succes(String)
    case erreur(String)
}
```

On peut alors écrire :

```
let resultat =
    ResultatTelechargement.succes(
        "photo.jpg"
    )
```

13. Récupérer la valeur associée

```
switch resultat {
case .succes(let fichier):
    print("Téléchargé : \(fichier)")
case .erreur(let message):
    print("Erreur : \(message)")
}
```

Le cas indique donc le type de situation, tout en transportant une information supplémentaire.

14. Exemple — État de téléchargement avec valeurs associées

```
enum EtatTelechargement {
    case attente
    case progression(Int)
    case termine(String)
    case erreur(String)
}
```

Exemples :

```
let etat1 =
    EtatTelechargement.attente
```

```swift
let etat2 =
    EtatTelechargement.progression(60)
let etat3 =
    EtatTelechargement.termine(
        "update.zip"
    )
```

15. Traiter les différents états

```swift
switch etat2 {
case .attente:
    print("En attente")
case .progression(let pourcentage):
    print("\(pourcentage)%")
case .termine(let fichier):
    print("Terminé : \(fichier)")
case .erreur(let message):
    print("Erreur : \(message)")
}
```

16. Une énumération peut avoir une méthode

```swift
enum EtatConnexion {
    case connecte
    case deconnecte

    func afficher() {
        switch self {
        case .connecte:
            print("Connecté")
        case .deconnecte:
            print("Déconnecté")
        }
    }
}
```

Utilisation :

```swift
let etat = EtatConnexion.connecte
etat.afficher()
```

17. Exercice — État d'une application

Créer :

```swift
enum EtatApplication
```

avec :

```
demarrage
active
arrierePlan
fermee
```

Créer une variable :

```
var etatApplication
```

Puis utiliser :

```
switch
```

pour afficher un message différent selon l'état.

18. Exercice — Résultat de connexion

Créer :

```
enum ResultatConnexion
```

avec :

```
case succes(String)
case erreur(String)
```

Exemples :

```
.succes("Alex")
```

et :

```
.erreur("Mot de passe incorrect")
```

Utiliser un switch pour afficher le résultat.

19. À retenir

Une énumération définit un ensemble limité de possibilités :

```
enum Etat {
    case actif
    case inactif
}
```

Elle fonctionne très bien avec :

```
switch
```

Elle peut utiliser des valeurs brutes :

```
enum Priorite: Int
```

ou transporter des valeurs associées :

```
case progression(Int)
case erreur(String)
```

Les enum Swift sont donc particulièrement utiles pour représenter :

- États
- Types
- Catégories
- Résultats
- Modes de fonctionnement

La prochaine présentation sera : Protocoles en Swift.